Milestone 1 Demo Script

AI Startup Idea Validator

(10-minute timed version)

Speakers: Yasaswini (opening / AI pipeline / repo walkthrough / closing), Varshini (AI frameworks / live demo), Yalene (backend), Anu Kumari (frontend)

Total runtime: ~10:00

---

1. Opening — Yasaswini

[TIME: 1:00] · **[VISUAL: title slide / landing page of the live site]**

"Good evening, everyone. I'm Yasaswini, from Team Nova. Today we're going to walk you through our project — Affinity.

So — normally, when a founder has a startup idea, before they even build anything, they need to research the market: who's the competition, is there real demand, what's already out there. That usually takes hours of manual searching.

What we built is a tool that does that automatically. A founder types in their idea, in a couple of lines, and our system goes and researches it live on the web, and comes back with a grounded market summary — real sources, not guesses. We called it Affinity, because that's exactly what it measures: does the market have an affinity for this idea, before you spend months building it.

For this milestone, we designed the full system architecture, built a working submission interface, and built a live web search agent that pulls real data — nothing here is hardcoded or faked. I'll hand over to Varshini to walk through the AI agent itself."

(Pause 1–2 sec before handing off — don't rush the transition.)

---

2. AI Agent — Frameworks — Varshini

[TIME: 1:15] · **[VISUAL: architecture diagram / agent code file, if available]**

"The core of this system is the Web Search Agent. We use two frameworks together, on purpose. **CrewAI** defines the agent itself: it has a role ('Web Search Agent'), a goal, and access to a search tool. **LangGraph** orchestrates the pipeline as a graph of steps. Right now there's one step — search — but it's built so every future milestone's agent, like Market Analysis or Competitor Discovery, just plugs in as a new step in the same graph, instead of us rebuilding the backend each time.

We made this choice deliberately, early on, because the project brief itself describes a multi-agent system — so we wanted the underlying structure to already match that shape, rather than retrofitting it later.

I'll pass it back to Yasaswini to walk through what actually happens when someone submits an idea."

---

3. AI Agent — Pipeline — Yasaswini

[TIME: 1:45] · **[VISUAL: pipeline/data-flow diagram, or the 5 research-angle labels on screen]**

"Right — so here's how a request actually flows. All of this runs as a single node inside the LangGraph pipeline Varshini just mentioned — when the graph reaches the `web_search` step, it does two things in parallel: it fetches real results in code, and it asks the CrewAI agent to summarize them. That's the step-by-step structure we'll be building every future milestone's agent into.

Within that step, when someone submits an idea, we expand it into **5 different research angles**: market size and trends, competitors, industry news, customer demand, and how other startups solve the same problem. Each angle runs its own search, so we get diverse, well-rounded coverage instead of one generic query.

For search, we use **Tavily** as the primary source, since it gives a real, trained relevance score. But we didn't want a single point of failure, so we built a free fallback chain — DuckDuckGo, Wikipedia, and Hacker News kick in automatically if Tavily's ever unavailable. We also filter out academic and research-paper sources, since those read as dense literature reviews rather than the market signal a founder actually needs.

One important design decision: the actual results you see are fetched **directly in code**, not generated by the LLM. The model's only job is writing a short summary on top of that real data — so sources are always genuine, never invented. We even added a safety check on the summary itself: if it ever looks like a broken or leaked reasoning trace, we automatically fall back to a safe, results-based summary instead of showing something broken.

Now I'll hand it to Yalene for the backend."

---

4. Backend & Deployment — Yalene

[TIME: 1:30] · **[VISUAL: swap to Render dashboard or the `/validate` response in devtools/Postman]**

"On the backend side, we built this with **FastAPI** — a lightweight Python framework that exposes one main endpoint, `POST /validate`. It receives the founder's idea, target customer, and

problem statement, runs it through the agent pipeline you just heard about, and returns a clean JSON response: a summary plus a list of result cards, each with a title, snippet, URL, which search angle it came from, and a relevance score.

We designed this with reliability in mind. If the idea field is empty, we return a clear `400` error immediately. If every search source fails, we return a `502` with a readable error message — never a raw stack trace. If a search comes back empty, the frontend shows a proper empty state instead of just looking broken.

For deployment, everything runs live on **Render** — as two separate services, one for the frontend and one for the backend, connected through environment variables. API keys are kept server-side and never committed to the repository. Both services auto-deploy whenever we push to our staging branch, so what we're showing you today is the actual live, deployed version — not just something running on a laptop."

---

5. Frontend & UX — Anu Kumari

[TIME: 1:15] · **[VISUAL: live site, scroll through the UI while talking]**

"For the interface, we used **React with Tailwind CSS**, and we deliberately avoided a generic template look. The design is a monochrome, technical-dashboard style — near-black background, monospace uppercase labels, and pill-style buttons.

A few details we're proud of: while the system is searching, we show a live status readout that cycles through real pipeline steps — 'Searching web,' 'Cross-checking sources,' 'Agent reasoning' — so the user always knows something real is happening, not just a spinner. Once results come back, they're grouped by research angle instead of being dumped as one long list, with a 'show more' option so it doesn't feel overwhelming. Each result card shows a relevance score that counts up on screen, and we added subtle hover motion on the cards so it feels responsive and alive.

We also handle every edge case in the UI: empty input shows an inline validation message instead of a browser alert, and any backend error shows a clear message with a retry button — the interface never just freezes or goes blank."

---

6. Live Demo — Varshini

[TIME: 1:45] · **[VISUAL: the live deployed site, full screen]**

(Switch to the live site. Submit a real idea and let each step visibly play out — don't rush past the loading state.)

"Let's see it live. I'll submit a real idea — 'A subscription box for eco-friendly cleaning products,' targeting environmentally conscious households, addressing plastic waste from cleaning product packaging.

(Submit. Let the status readout actually cycle on screen for a few seconds.)

You can see the live status updating as it works — searching the web, cross-checking sources, agent reasoning — this isn't a fake spinner, it's reflecting real steps.

(Once results load.)

And here's the summary, grounded in the actual sources below it. Results are grouped by research angle — here's the market-size angle, here's competitors — and every card has a real link. I'll click one to show it's a genuine, current source, not something we typed in ourselves."

(Optionally click one result link to open it briefly, then return to the app.)

---

7. GitHub Repo Walkthrough — Yasaswini

[TIME: 0:45] · **[VISUAL: switch tabs to the GitHub repository]**

"Before we wrap up, quickly — this is all in our GitHub repo, so everything we've described is verifiable, not just slides.

(Scroll the repo root.)

You can see the folder structure matches what we described: `frontend/` for the React app, `agent/` or `backend/` for the FastAPI service and search pipeline, and `docs/` where we keep the architecture doc and the API contract we locked on day one, so the team could build in parallel without blocking each other.

(Optionally open `docs/architecture.md` *briefly.)*

This is also where our commit history lives — you can see this was built incrementally over the milestone, not thrown together the night before."

---

8. Why We Chose This Stack + Conclusion — Yasaswini

[TIME: 0:45] · **[VISUAL: back to the live site or a closing slide]**

"To quickly justify our choices: we picked **CrewAI and LangGraph together** because the project brief itself describes a multi-agent pipeline — so future agents just plug into the same structure instead of us rebuilding the backend each time. We picked **Tavily with a free fallback** because we didn't want the whole project blocked on one API key or one provider's uptime. And we chose **React and Tailwind** because we wanted this to feel like a real product, backed by real engineering: validation, error handling, and a quality gate on the AI's own output.

To summarize: for Milestone 1, we delivered a documented system architecture, a working idea submission interface, and a live web search agent — all deployed and functioning end-to-end

today, not just on paper. And it's structured so Milestones 2 through 4 — Market Analysis, Competitor Discovery, SWOT, MVP Recommendation, and Report Generation — plug into this same pipeline without needing a rewrite.

Thank you — happy to answer any questions."

Quick Q&A Prep

- **"Why not just use Tavily alone?"** → Reliability. A single point of failure on one teammate's API key isn't acceptable for a live demo or production use.

- **"Why CrewAI/LangGraph for just one agent?"** → We're paying that setup cost once now so every future agent is a small addition, not a rewrite.

- **"Is the data really live?"** → Yes — click any result link on screen; it's a real, current source, not fabricated.

- **"What happens if the AI makes something up?"** → It can't for the results (fetched directly in code), and the summary itself is validated before being shown.

Rehearsal Checklist

- [ ] Have the live deployed site open and logged in/ready **before** you start
- [ ] Have the GitHub repo tab already open in the background (don't waste time navigating to it live)
- [ ] Pre-type or copy-paste the demo idea text so you're not typing live on stage
- [ ] Do one full run-through with a stopwatch — if you're over 10:30, trim adjectives in the pipeline and backend sections first, not the live demo
- [ ] Confirm handoff lines out loud with the whole team at least once before presenting